

ProdNet: a Lightweight Network for Fast Discovery of Matrix Multiplication Algorithms

Anonymous Authors¹

Abstract

Matrix multiplication is a fundamental operation with applications across various fields, including physics, electronics, and artificial intelligence. However, its standard implementation exhibits cubic time complexity, posing computational challenges, especially in deep learning algorithms requiring large-scale matrix operations. We propose ProdNet, a lightweight neural network model that discovers matrix multiplication algorithms without requiring extensive computational resources or any prior knowledge about these algorithms. Our model aims to reduce the complexity by minimizing the number of multiplications in the multiplication. We employ a combination of MSE and regularization function as we target the weight values to be 0, 1, or -1. We will demonstrate, in this study that ProdNet discovers multiplication algorithms for 2×2 and 3×3 matrices size, achieved in a few iterations. We highlight the effectiveness of our regularization techniques in enhancing the speed of algorithm discovery.

1. Introduction

Matrix multiplication is a foundational operation in mathematics and machine learning, serving as a cornerstone in various computational tasks, including the training of deep learning models (Alsari & Al-Hashimi, 2024). It plays a crucial role in optimization procedures, such as parameter adjustments in neural networks (Goodfellow et al., 2016), enabling the refinement of model parameters for improved performance. Moreover, matrix multiplication facilitates essential operations like feature extraction and transformation (Bishop, 2006), contributing to the extraction of meaningful patterns from data.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

In deep learning, matrices form the backbone of neural network architectures. These models, comprising interconnected layers of neurons, conduct operations on input data represented as matrices. The efficiency of matrix multiplication is paramount for the performance and scalability of deep learning models, given that both forward and backward propagation heavily rely on these operations (LeCun et al., 1998).

The standard matrix multiplication (SMM) algorithms often exhibit cubic time complexity. When multiplying two $n \times n$ matrices, A and B, to obtain an $n \times n$ product matrix, C, the process involves computing the dot product of each row of A with each column of B. Each of these dot products requires n multiplications and $n - 1$ additions, resulting in a total of n^3 multiplications and $n^2(n - 1)$ additions (Li et al., 2020). This leads to prolonged computation times, especially when dealing with large matrices in machine learning models. These limitations are particularly pronounced in scenarios involving large matrices, where computational complexity may escalate to prohibitive levels.

Many large deep learning models, such as Long-Short Term Memory (LSTM) networks, large language models (LLMs), and Transformers, rely heavily on the standard matrix multiplication algorithm (SMM) during their training. This reliance imposes significant computational demands, requiring substantial hardware accelerators, which in turn necessitate considerable financial investment (Menghani, 2023). Although, these models have demonstrated remarkable performance in automating human tasks and serving as intelligent agents across various domains, driving further advancements in hardware accelerators, but recent studies (Anthony et al., 2020; Budenny et al., 2022; Huguerte et al., 2023) have highlighted two major issues: the environmental impact and the financial burden of training deep learning models on advanced machines:

Environmental Impact: Training large deep learning models requires immense computational power, often provided by data centers equipped with numerous GPUs and TPUs. These data centers consume substantial amounts of electricity, which, depending on the energy sources, can lead to significant carbon emissions. This high energy consumption contributes to the carbon footprint of artificial intelligence,

exacerbating climate change and environmental degradation, which goes against the principles of sustainable development.

Financial Burden: Developing and training these sophisticated models also demands substantial financial resources. The costs associated with acquiring and maintaining cutting-edge hardware, such as GPUs and TPUs, are high. Furthermore, the electricity needed to power these machines adds to the operational costs. This financial burden is particularly challenging for research labs and institutions with limited budgets, potentially hindering progress and innovation in the field and limiting its democratization.

Consequently, researchers are exploring strategies to mitigate these challenges by reducing the computational complexity of training (Menghani, 2023). One promising approach involves the adoption of novel matrix multiplication algorithms with fewer operations. These algorithms aim to make the training process more efficient, thereby reducing both the environmental impact and the financial costs associated with developing and deploying deep learning models. By improving the efficiency of matrix multiplication, researchers hope to create more sustainable and cost-effective solutions for training advanced deep learning models.

In 1969, the mathematician Volker Strassen proposed an algorithm capable of multiplying 2×2 matrices with only 7 multiplication steps, albeit with additional addition steps. Strassen’s insight that matrices themselves can consist of matrix elements allowed for its adaptation to larger matrices (Strassen et al., 1969). By recursively applying this technique, notable efficiency enhancements were achieved, theoretically reducing the time complexity of matrix multiplication from $O(n^3)$ to approximately $O(n^{\log_2(7)}) = O(n^{2.81})$ (Strassen et al., 1969). Despite its theoretical benefits, Strassen’s algorithm is not commonly used directly in deep learning models. However, it has inspired research in finding low-cost approximations of matrix multiplication.

In this paper, we introduce ProdNet, a lightweight network model tailored for discovering efficient matrix multiplication algorithms. Our primary objective is to expedite this process by adopting the right loss function during training. Our loss function is a combination of Mean Squared Error (MSE) and regularization terms that have been chosen carefully so that they minimize the number of multiplications in the algorithm discovered by the network. In addition, we compare the used training loss functions for fast convergence. The main contributions of this study:

- First, we introduce, ProdNet, a flexible lightweight network model tailored for discovering matrix multiplication algorithms.
- Second, we introduce regularization function to accelerate the discovery of model convergence and multi-

cation algorithm discovery.

2. Related Works

Strassen’s breakthrough prompted the exploration of efficient matrix multiplication algorithms, giving rise to two distinct subfields. The first subfield examines the scaling behavior of multiplication steps as matrix dimensions approach infinity. Notably, Coppersmith and Winograd devised a modification of Strassen’s sequential algorithm, leading to further enhancement in computational efficiency with a runtime of $O(n^{2.375})$ (Coppersmith & Winograd, 1987). Following this, Virginia Williams, affiliated with Stanford University, introduced the most efficient Strassen’s algorithm to date, with a runtime of $O(n^{2.373})$ (Williams, 2014). However, these algorithms primarily serve theoretical purposes, applicable only to exceedingly large matrices.

The second subfield focuses on smaller-scale matrix operations, particularly those involving 2×2 matrices. Winograd (Winograd, 1971) established a theoretical limit of 7 multiplication steps for such matrices, highlighting the fundamental complexity involved. Despite this advancement, determining minimal requirements for larger matrix sizes remains an unresolved challenge within the field. Developing algorithms optimized for smaller matrices holds promise for practical applications, potentially outperforming Strassen’s algorithm in real-world scenarios.

Researchers of (Elser, 2016) and (Fawzi et al., 2022) have endeavored to address this challenge by re-imagining matrix multiplication as a distinct mathematical problem, one that offers improved computational tractability. They represent the abstract task of multiplying two matrices as a three-dimensional array of numbers known as a tensor. Researchers then decompose this tensor into a sum of elementary components, referred to as ‘rank-1’ tensors, each representing a distinct step in the corresponding matrix multiplication algorithm. Consequently, finding an efficient multiplication algorithm involves minimizing the number of terms in tensor decomposition; fewer terms result in fewer steps required.

In 2022, Google’s DeepMind team introduced AlphaTensor, a model designed to discover efficient matrix multiplication algorithms. Leveraging reinforcement learning techniques inspired by AlphaZero (Silver et al., 2018). AlphaTensor utilizes a deep neural network architecture based on attention mechanisms (Vaswani et al., 2017) to guide Monte Carlo tree search (MCTS) algorithms. AlphaTensor demonstrated remarkable capabilities in discovering novel matrix multiplication algorithms, uncovering numerous existing algorithms, and proposing new ones that outperformed existing approaches by reducing the number of required multiplications.

However, training the model required a significant amount of multiplication algorithm data, prompting the team to implement data augmentation techniques to generate additional training samples from completed games. The DeepMind research group endeavored to train AlphaTensor in the task of tensor decomposition representing the product of matrices up to dimensions of 12×12 . This pursuit involved the development of expedient algorithms for matrix multiplication operations involving conventional real number elements, as well as algorithms tailored to the specialized domain of modulo 2 arithmetic. The latter pertains to a constrained mathematical framework wherein computations are conducted exclusively with two numerical values, typically denoted as 0 and 1. Despite its promising results, training AlphaTensor posed several challenges. Reinforcement learning models, such as AlphaTensor, are notoriously difficult to train due to the complexities involved in setting reward and penalty parameters effectively (Ji et al., 2023). Moreover, the large number of parameters in AlphaTensor necessitated substantial computational resources, making it inaccessible to many research labs.

AlphaTensor possesses a notable advantage over established computer search methods, yet this advantage is also its primary limitation. Unlike traditional methods, AlphaTensor operates without the constraints of human intuition regarding optimal algorithmic structures. Consequently, it lacks the capacity to elucidate the rationale behind its decisions, hindering researchers' ability to glean insights from its accomplishments. Shortly after AlphaTensor's breakthrough results, mathematician Manuel Kauers and graduate student Jakob Moosbauer introduced a novel approach (Kauers & Moosbauer, 2022). Their method involves iteratively refining existing algorithms to achieve further efficiency gains. Starting with AlphaTensor's algorithm for 5×5 modulo 2 matrices as a foundation, they observed a reduction in the number of required multiplication steps from 96 to 95 within mere seconds of computation. In (Elser, 2016), the author explores neural networks with a unique twist: their only non-linear components are multipliers. The goal is to test a novel training rule in a context where precise data representation is paramount. These networks are specifically challenged to discover the rules of matrix multiplication, with a focus on Strassen multiplication. The only training protocol employed is called conservative learning, which ensures minimal weight adjustments to produce the correct output for each input as soon as the input-output pair is received. The neural network designed in this study is intriguing due to its minimalistic design, containing only seven multipliers, mirroring the seven scalar multiplications used in Strassen's algorithm for 2×2 matrices. However, it is important to note that this conservative learning technique does not accelerate the convergence of the model, and the discovery of the algorithms requires large numbers

of epochs. Also, the authors here did not compare it to any other training techniques, which limited the insights their work provided into the theory of improving matrix multiplication algorithms discovery.

Motivated by this potential, our study introduces ProdNet, a lightweight network model for the discovery of novel matrix multiplication algorithms. Our primary objective is to develop a trainable model that operates efficiently on modest hardware setups. By leveraging the algorithms uncovered by ProdNet, we aim to accelerate the training process of deep learning models, ultimately facilitating faster convergence and improved efficiency in AI applications.

3. Multiplication Algorithm and Tensor Decomposition

As previously discussed, contemporary research has reconceptualized matrix multiplication as a mathematical problem conducive to computational processing. The matrix multiplication operation can be represented as a 3D tensor (Pan, 1804). Denoting T_n as the tensor describing the $N \times N$ matrix multiplication, this 3D tensor comprises binary values, where 0 indicates no multiplication and 1 denotes multiplication.

The T_n tensor can be decomposed into an outer product ($\otimes$) of rank-1 tensors, expressed as $T_n = \sum_{r=1}^R u^r \otimes v^r \otimes w^r$, where R denotes the number of vectors, akin to the number of multiplications involved, and each u , v , and w is a rank-one vector (Fawzi et al., 2022). These stacked rank-one vectors constitute a rank- R decomposition of the 3D tensor, which readily translates into a matrix multiplication algorithm.

The multiplication of 2×2 matrices can be performed with 7 multiplication instead of 8 by following Strassen's algorithm.

$$a = \begin{pmatrix} a_1 & a_2 \\ a_3 & a_4 \end{pmatrix} \quad b = \begin{pmatrix} b_1 & b_2 \\ b_3 & b_4 \end{pmatrix} \quad c = \begin{pmatrix} c_1 & c_2 \\ c_3 & c_4 \end{pmatrix}$$

$$U = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & -1 \end{pmatrix}$$

$$V = \begin{pmatrix} 1 & 1 & 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & -1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

$$W = \begin{pmatrix} 1 & 0 & 0 & 1 & -1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & -1 & 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

This decomposition can be translated to a multiplication algorithm as follows, where the matrices' values of $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{W}$ are special for a , b , and c respectively.

$$m_1 = (1a_1 + 1a_4) \cdot (1b_1 + 1b_4)$$

$$m_2 = (1a_3 + 1a_4) \cdot (1b_1)$$

$$m_3 = (1a_1) \cdot (1b_2 - 1b_4)$$

$$m_4 = (1a_4) \cdot (1b_3 - 1b_1)$$

$$m_5 = (1a_1 + 1a_2) \cdot (1b_4)$$

$$m_6 = (1a_3 - 1a_1) \cdot (1b_1 + 1b_2)$$

$$m_7 = (1a_2 - 1a_4) \cdot (1b_3 + 1b_4)$$

$$c_1 = 1m_1 + 1m_4 - 1m_5 + 1m_7$$

$$c_2 = 1m_3 + 1m_5$$

$$c_3 = 1m_2 - 1m_4$$

$$c_4 = 1m_1 - 1m_2 + 1m_3 + 1m_6$$

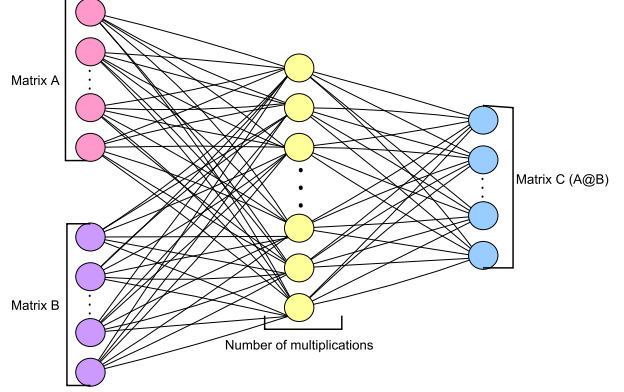


Figure 1. ProdNet Architecture.

4.1. Training Loss

The objective of our training process is to achieve accurate matrix multiplication while constraining the weights to be either 0, 1, or -1. To achieve this, we design a loss function combining mean squared error (MSE) with a regularization term:

$$Loss = MSE + \frac{\alpha}{M} \sum_x \sum_{n=1}^N l_0 \times l_1 \times l_{-1}, \quad (1)$$

where $l_i = |w_n^x - i|$, $\forall x \in \{A, B, C\}$, $\forall i \in \{-1, 0, 1\}$

4. Methodology

In order to discover multiplication algorithms, we designed a lightweight neural network. Although the architecture of our network is similar to the one in (Elser, 2016), but the way we discover multiplication algorithms is completely different. They employ a conservative learning rule to adjust the weights minimally. In contrast, our method uses regularization to enforce specific weight values 0, 1, or -1. In this setup, the neural network's trained weights correspond to the factors of the matrices $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{W}$. This methodology offers flexibility in determining the desired number and shape of matrices used for training, enhancing the adaptability of the training process to various scenarios.

The architecture of the network is illustrated in Figure 1. We denote by $\mathbf{W}^A$ the weight matrix between the input layer A and the hidden layer and $\mathbf{W}^B$ is the weight matrix between the input layer B and the hidden layer. $\mathbf{W}^C$ denotes the weight matrix between the hidden layer and the output C. In the model, $(\mathbf{W}^A)^\top$, $(\mathbf{W}^B)^\top$, and $\mathbf{W}^C$ emulates the $\mathbf{U}$, $\mathbf{V}$, and $\mathbf{W}$ matrices, respectively. The activation function f of each neuron of the hidden layer is defined as the dot product, $f = a \cdot b$, where $a = A(\mathbf{W}^A)^\top$ and $b = B(\mathbf{W}^B)^\top$. The output of the hidden layer g is multiplied by the weight matrix $\mathbf{W}^C$, yielding the output $C = g(\mathbf{W}^C)^\top$.

α is the penalty set to the regularization function, M is the total number of parameters, and N is the number of parameters in the matrix. The MSE loss measures the disparity between the predicted and actual outputs, guiding the improvement of the model's efficiency and targeting the correct values of the multiplication.

Simultaneously, the regularization term penalizes deviations of the weights from the desired values (0, 1, or -1). Initially, we employed a multiplication-based regularization approach. The primary concept behind this regularization is that using the product of norms allows the smallest norm value to dominate the result, thereby guiding the weight values towards one of the desired values (0, 1, or -1). However, one of the ultimate goals of training is to obtain a pruned model, meaning a model with a large number of weights set to 0, to reduce the number of operations. Using the product of norms does not allow for setting penalties on specific l_0 norms independently. This limitation motivated us to develop a similar regularization function that maintains performance while allowing us to specifically penalize the l_0 norm.

After several training epochs, we implement a round-to-integer operation to ensure that learned parameters converge to the desired discrete values (-1, 0, or 1), further optimizing our model's performance.

5. Results

Our study focuses on the development of a lightweight neural network, ProdNet, designed to discover matrix multiplication algorithms using efficient and computationally simple learning techniques. ProdNet is both easy to train and minimally resource-intensive. To evaluate its performance and assess the impact of the regularization function, we conducted multiple training sessions on 2×2 matrices with either 7 multiplications or 8 multiplications, as well as conducting training sessions on 3×3 matrices with 23 multiplications. All the results was discovered using Tesla T4 GPU (in Google collab and Kaggle clouds).

Initially, we trained the model using only the mean squared error (MSE) as the loss function to validate the necessity of incorporating a regularization term. The training process utilized a dataset of 8000 samples, with weight initialization performed using the Xavier uniform distribution and optimization achieved using the Adam optimizer with a learning rate of 0.05. Our model demonstrated potential during training, as shown in Figure 2, which illustrates the model’s convergence when MSE is used without rounding weight values to integers.

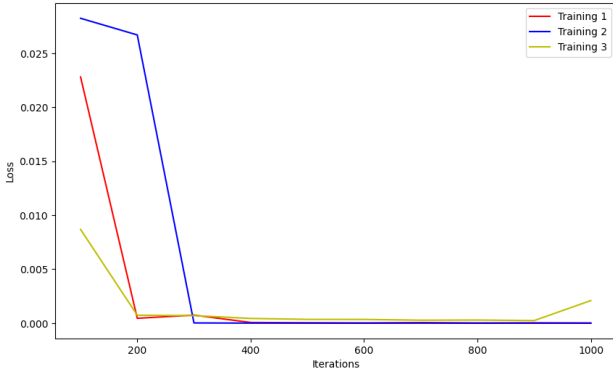


Figure 2. Model Convergence During Training Using MSE Without Integer Rounding of Weights.

However, due to the requirement for the weight values to be restricted to discrete integers (0, 1, and -1) to reduce the number of multiplications, continuous weight values are not acceptable. This constraint necessitated rounding the trained weights to the nearest integers post-training. Unfortunately, this rounding process often degraded the model’s performance, resulting in suboptimal solutions and hindering the discovery of accurate multiplication algorithms. Figure 3 illustrates the model’s convergence when rounding to integers is applied.

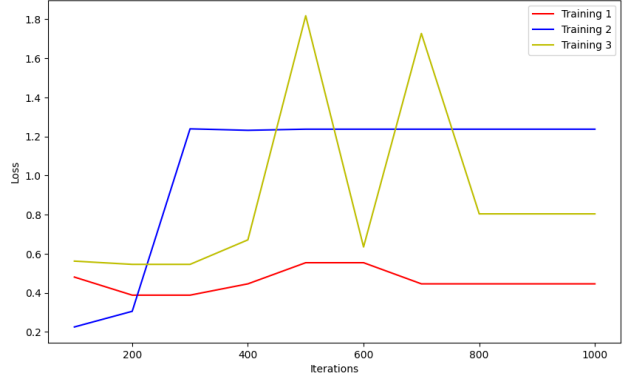


Figure 3. Model Convergence During Training Using MSE With Integer Rounding of Weights.

In the second phase, we conducted multiple training sessions on 2×2 matrices, utilizing the defined regularization function in conjunction with the Mean Squared Error (MSE) as the loss function. The results demonstrated that ProdNet could easily discover multiplication algorithms requiring 8 multiplications. However, to focus on discovering algorithms with a reduced number of multiplications, we choose the network to have 7 perceptrons in the hidden layers. This configuration ensured that the number of multiplications remained within the best-known lower rank for 2×2 matrices. The training process utilized a dataset consisting of 8000 samples of paired matrices and their corresponding multiplication results, processed as a full batch. For weight initialization, we employed the Xavier uniform distribution. Additionally, the hyperparameter $\alpha = 1/500$ was incorporated into the loss function to guide the optimization process effectively, optimizing the training using the Adam optimizer. Figure 4 illustrates five samples of training launched using different weight initialization values and learning rates. Where after each training session, a different and unique multiplication algorithm was discovered. Some of the discovered algorithms:(Check A for the rest).

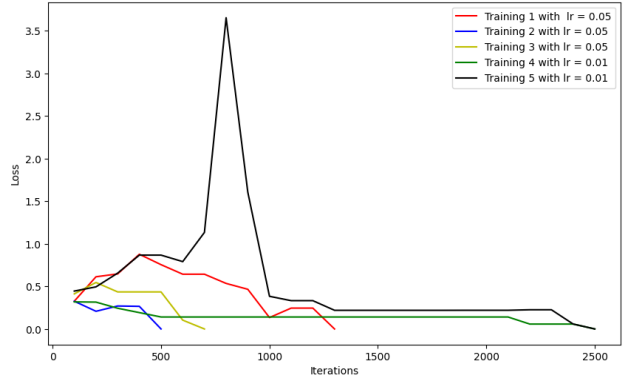


Figure 4. model convergence using different weight initialization values and learning rate.

The algorithm was discovered in 500 iterations.

$$\begin{aligned}
 m_1 &= (-a_1 + a_3) \cdot (-b_3 + b_4) \\
 m_2 &= (-a_1 - a_2) \cdot (-b_2) \\
 m_3 &= (-a_3 - a_4) \cdot (b_3) \\
 m_4 &= (a_2) \cdot (-b_2 + b_4) \\
 m_5 &= (-a_2 - a_3) \cdot (b_2 - b_3) \\
 m_6 &= (-a_1 + a_3) \cdot (-b_1 + b_2) \\
 m_7 &= (a_3) \cdot (-b_1 + b_3) \\
 c_1 &= m_2 + m_5 + m_6 - m_7 \\
 c_2 &= m_2 + m_3 \\
 c_3 &= -m_3 - m_7 \\
 c_4 &= -m_1 - m_3 + m_4 - m_5
 \end{aligned}$$

The algorithm was discovered in 2500 iterations.

$$\begin{aligned}
 m_1 &= (a_1 + a_2) \cdot (-b_1) \\
 m_2 &= (-a_2 - a_4) \cdot (b_3 - b_4) \\
 m_3 &= (a_2 - a_3) \cdot (b_1 - b_4) \\
 m_4 &= (-a_1 - a_3) \cdot (b_1 - b_2) \\
 m_5 &= (a_3) \cdot (-b_2 + b_4) \\
 m_6 &= (a_2) \cdot (b_1 - b_3) \\
 m_7 &= (a_3 + a_4) \cdot (-b_4) \\
 c_1 &= -m_1 - m_6 \\
 c_2 &= -m_1 - m_3 + m_4 + m_5 \\
 c_3 &= -m_2 - m_3 + m_6 - m_7 \\
 c_4 &= -m_5 - m_7
 \end{aligned}$$

For 3×3 matrices, our model was trained to discover a multiplication algorithm requiring 23 multiplications instead of the standard 27. To achieve this, the network was designed with 23 perceptrons in the hidden layers. The training process utilized a dataset of 8000 samples, with weights initialized using the Xavier uniform distribution. Optimization was performed using the Adam optimizer with a learning rate of 0.001, and the hyperparameter $\alpha = 1/50$ was applied to the regularization in the loss function. Figure 5 illustrates that our model was able to discover the multiplication algorithm in 8800 iterations.

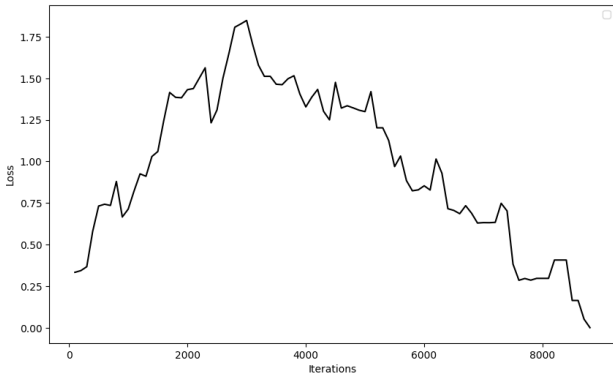


Figure 5. model convergence for 3 by 3 matrix size

The algorithm discovered for 3 by 3 matrices size.

$$\begin{aligned}
 m_1 &= (-a_9) \cdot (b_8 - b_9) \\
 m_2 &= (a_7 - a_8) \cdot (-b_3) \\
 m_3 &= (a_6) \cdot (b_5 + b_8) \\
 m_4 &= (a_1 + a_3 - a_4 + a_7) \cdot (b_1 + b_2 + b_4 - b_7) \\
 m_5 &= (-a_1 - a_3 + a_4 + a_6 - a_7 - a_9) \cdot (-b_7) \\
 m_6 &= (-a_1 + a_3) \cdot (b_9) \\
 m_7 &= (-a_6) \cdot (b_1 - b_3 - b_4 + b_6 - b_7 + b_9) \\
 m_8 &= (a_2 - a_3 + a_8) \cdot (-b_5 - b_9) \\
 m_9 &= (a_3) \cdot (-b_1 - b_2 - b_4 + b_5 + b_7 + b_8) \\
 m_{10} &= (-a_4 + a_7) \cdot (b_2) \\
 m_{11} &= (a_1 + a_3) \cdot (-b_1 - b_2 - b_4) \\
 m_{12} &= (-a_4 + a_5 + a_7 - a_8) \cdot (b_4) \\
 m_{13} &= (-a_1 - a_8) \cdot (b_3 + b_4) \\
 m_{14} &= (-a_5 + a_6) \cdot (b_4 - b_6) \\
 m_{15} &= (a_1 - a_4 - a_6 + a_7) \cdot (-b_1 + b_3 + b_7) \\
 m_{16} &= (a_2 + a_8) \cdot (-b_4 + b_6 + b_9) \\
 m_{17} &= (-a_1 + a_4 - a_7) \cdot (-b_1 - b_4 + b_7) \\
 m_{18} &= (-a_4 - a_6) \cdot (b_1 - b_3) \\
 m_{19} &= (a_1 - a_2) \cdot (b_4) \\
 m_{20} &= (-a_7) \cdot (-b_3 + b_5 - b_6) \\
 m_{21} &= (a_5 - a_6) \cdot (b_5) \\
 m_{22} &= (-a_7) \cdot (-b_2) \\
 m_{23} &= (-a_2 + a_3) \cdot (b_5)
 \end{aligned}$$

$$\begin{aligned}
 c_1 &= -m_4 + m_{10} + m_{17} - m_{19} \\
 c_2 &= m_4 + m_9 - m_{10} - m_{17} - m_{23} \\
 c_3 &= m_8 - m_{13} + m_{16} - m_{19} - m_{20} - m_{23} \\
 c_4 &= -m_2 + m_{12} - m_{13} - m_{15} - m_{17} - m_{18} \\
 c_5 &= m_3 - m_{10} + m_{21} + m_{22} \\
 c_6 &= m_2 - m_7 + m_{12} - m_{13} + m_{14} - m_{15} - m_{17} \\
 c_7 &= -m_2 + m_4 + m_5 - m_{10} + m_{11} - m_{13} - m_{15} - m_{17} - m_{18} \\
 c_8 &= -m_1 + m_6 - m_8 + m_{22} + m_{23} \\
 c_9 &= -m_2 + m_6 - m_8 + m_{20} + m_{23}
 \end{aligned}$$

The primary strength of our model lies in its simplicity and flexibility, enabling the discovery of multiplication algorithms for any specified matrix size using supervised learning. Furthermore, ProdNet does not require a large dataset for training, as the data can be easily generated, and it achieves algorithm discovery within a small number of iterations. These characteristics, coupled with minimal computational resource requirements, make ProdNet an efficient and accessible solution.

In contrast, AlphaTensor, as described in (Fawzi et al., 2022), employs deep reinforcement learning and relies on a well-known but rare mathematical dataset of multipli-

cation algorithms. To address data scarcity, AlphaTensor requires data augmentation techniques. Its complex architecture integrates Monte Carlo Tree Search (MCTS) with a Transformer-based neural network, necessitating training on a combination of input data, policy targets, and value targets. This complexity significantly increases training time, demanding extensive computational resources such as TPUs and several days to converge.

6. Conclusion

This paper highlights the critical role of matrix multiplication in various computational tasks and acknowledges the limitations of the standard algorithms, which require a large number of multiplications. The existing works for this task require huge computational resources and large multiplication algorithms datasets for the training, this motivates the exploration of innovative solutions to reduce this need for materials. We introduce ProdNet, a lightweight network model designed to efficiently and automatically discover matrix multiplication algorithms without prior knowledge. Our model successfully discovers algorithms for 2×2 matrices with only 7 multiplications and 3×3 with only 23 multiplications, the known lower-rank number, by employing a combination of MSE and regularization function. Using ProdNet emerges as a promising approach, demonstrating significant efficiency improvements while remaining computationally cost-effective. Notably, the model independently rediscovers numerous algorithms after every training launched, with the same or different parameters, highlighting its potential to address computational challenges in deep learning applications.

Regardless of the obtained promising results, there are some advancements to be improved. One avenue for improvement involves extending the size of the matrices such as 4×4 . As well as the application of regularization to additional values, such as -2 and 2. Additionally, future developments aim to advance the model's capabilities to discover the optimal multiplication algorithm by maximizing the occurrence of rank-one vectors with zero values in the U and/or V matrices, thereby automatically reducing the number of multiplications. Exploring discreet optimizers and using other search techniques a modified version of the exhaustive search technique to discover all the possible multiplication algorithms in a reduced search space instead of $3^{(\text{number of model's parameters})}$.

References

Alsari, S. and Al-Hashimi, M. Investigation of energy and power characteristics of various matrix multiplication algorithms. *Energies*, 17(9):2225, 2024.

Anthony, L. F. W., Kanding, B., and Selvan, R. Car-

bontracker: Tracking and predicting the carbon footprint of training deep learning models. *arXiv preprint arXiv:2007.03051*, 2020.

Bishop, C. M. Pattern recognition and machine learning. *Springer google schola*, 2:5–43, 2006.

Budenny, S. A., Lazarev, V. D., Zakharenko, N. N., Korovin, A. N., Plosskaya, O., Dimitrov, D. V., Akhrikin, V., Pavlov, I., Oseledets, I. V., Barsola, I. S., et al. Eco2ai: carbon emissions tracking of machine learning models as the first step towards sustainable ai. In *Doklady Mathematics*, volume 106, pp. S118–S128. Springer, 2022.

Coppersmith, D. and Winograd, S. Matrix multiplication via arithmetic progressions. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pp. 1–6, 1987.

Elser, V. A network that learns strassen multiplication. *The Journal of Machine Learning Research*, 17(1):3964–3976, 2016.

Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatin, M., Novikov, A., R Ruiz, F. J., Schrittwieser, J., Swirszcz, G., et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

Goodfellow, I., Bengio, Y., and Courville, A. *Deep learning*. MIT press, 2016.

Heguer, L. B., Bugeau, A., and Lannelongue, L. How to estimate carbon footprint when training deep learning models? a guide and review. *arXiv preprint arXiv:2306.08323*, 2023.

Ji, J., Qiu, T., Chen, B., Zhang, B., Lou, H., Wang, K., Duan, Y., He, Z., Zhou, J., Zhang, Z., et al. Ai alignment: A comprehensive survey. *arXiv preprint arXiv:2310.19852*, 2023.

Kauers, M. and Moosbauer, J. The fbhhrbnrsshk-algorithm for multiplication in $\mathbb{Z}_{2^5}$ is still not the end of the story. *arXiv preprint arXiv:2210.04045*, 2022.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

Li, Y., Hu, S.-L., Wang, J., and Huang, Z.-H. An introduction to the computational complexity of matrix multiplication. *Journal of the Operations Research Society of China*, 8:29–43, 2020.

Menghani, G. Efficient deep learning: A survey on making deep learning models smaller, faster, and better. *ACM Computing Surveys*, 55(12):1–37, 2023.

- Pan, V. Fast feasible and unfeasible matrix multiplication, april 2018, 1804.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- Strassen, V. et al. Gaussian elimination is not optimal. *Numerische mathematik*, 13(4):354–356, 1969.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Williams, V. V. Multiplying matrices in $O(n^2 \cdot 373)$ time. *preprint*, pp. 0–105698, 2014.
- Winograd, S. On multiplication of 2×2 matrices. *Linear algebra and its applications*, 1971.

A. You *can* have an appendix here.

You can have as much text here as you want. The main body must be at most 8 pages long. For the final version, one more page can be added. If you want, you can use an appendix like this one.

The `\onecolumn` command above can be kept in place if you prefer a one-column appendix, or can be removed if you prefer a two-column appendix. Apart from this possible change, the style (font size, spacing, margins, page numbering, etc.) should be kept the same as the main body.